

Relatório de Programação Concorrente e Distribuída

Ana L. Tomita¹, Fabricio M. Muller¹, Gabriel A. Garcia¹,
Gabriel V. F. do Carmo¹, Loren P. R. Lorenzetto¹.

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (Unifesp)

{gabriel.almeida12,rafael.lorenzetto,gabriel.fabri,tomita.ana,fmmuller
}@unifesp.br

Resumo. [texto]

1. Introdução

[texto]

2. Fundamentação Teórica

2.1. Computação Paralela

Do período de 1986 até 2002, o desempenho dos processadores aumentava cerca de 52% a cada ano [Herlihy and Shavit 2008]. No entanto, desde 2002 o crescimento de desempenho das novas gerações de microprocessadores tem desacelerado cada vez mais, isso se deve ao limite físico do silício [Hennessy and Patterson 2017]. Desse modo, por volta de 2005, a grande maioria das fabricantes de microprocessadores decidiu que o caminho para o ganho de desempenho estava direcionado à programação paralela [Pacheco 2011].

[Pacheco 2011] aponta que na computação paralela "um programa é aquele no qual múltiplas tarefas cooperam intimamente para resolver um problema". Nesse sentido, operações matriciais representam um problema adequado para o paralelismo, já que o cálculo de elementos diferentes podem ser distribuídos em diferentes unidades de execução.

2.2. Multiplicação de Matrizes

De acordo com [Cormen et al. 2012], o problema clássico para multiplicação de matrizes envolve duas matrizes dadas, A e B, e C uma nova matriz a ser calculada. A operação que define cada elemento C_{ij} pode ser dada como:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj} \quad (1)$$

Ou seja, cada elemento C_{ij} é obtido pelo produto escalar entre a linha i de A e a coluna j de B. Dessa forma, o algoritmo sequencial possui um custo $O(n^3)$ para matrizes de dimensão N por N.

2.3. Vetorização

Segundo [Allen and Kennedy 2002] vetorização foi uma das técnicas que permitiram com que resultados ideais em diversas arquiteturas paralelas ocorressem para multiplicação de matrizes. A vetorização é uma técnica que permite explorar o paralelismo a nível de dados

por meio de instruções SIMD (*Single Instruction, Multiple Data*), ou seja, operação de múltiplos dados sobre a mesma instrução [Hennessy and Patterson 2017].

Na vetorização automática o compilador identifica trechos específicos do código como laços de repetição, em que as operações podem ser realizadas de forma paralela em nível de instrução, e os transformam em instruções vetoriais compatíveis com os recursos do processador. Dessa forma, é possível reduzir o número de instruções para processar um conjunto de dados, melhorando o tempo de execução de programas, permitindo com que a multiplicação de matrizes seja executado em tempo ideal [Intel Corporation 2023].

2.4. Princípio da localidade espacial

[Hennessy and Patterson 2017] mostram que segundo o princípio da localidade espacial, quando um dado é acessado, existe uma grande possibilidade dos dados armazenados próximos serem necessários. Dessa forma, quando um dado não é encontrado em *cache*, o processador busca um bloco de dados próximos ao dado na memória principal.

Nesse sentido, segundo [Cplusplusreference.com 2026], em C/C++ as matrizes multidimensionais são armazenadas em ordem *row-major*, ou seja, os elementos de uma mesma linha ocupam posições consecutivas na memória. Por conseguinte, percorrer os elementos de uma linha de forma consecutiva, tende a favorecer a localidade espacial, já percorrer por coluna desfavorece o aproveitamento de dados em *cache*.

2.5. Transposição de matrizes

No cenário de multiplicação de matrizes o acesso à memória da matriz B terá um custo maior que A. Assim, a transposição da matriz, definido por [Cormen et al. 2012] como a permutação dos elementos de cada i -ésima linha, pela j -ésima coluna, pode resolver o problema do armazenamento na memória, alterando apenas a forma de cálculo da matriz C para manter o resultado, computando com os elementos da linha " j " ao invés da coluna.

2.6. Algoritmo de Strassen para multiplicação de matrizes

Segundo [Ikenmeyer and Lysikov 2019] o algoritmo de Strassen para multiplicação de matrizes foi uma descoberta revolucionária na álgebra linear computacional. A ideia central desse algoritmo é multiplicar matrizes de tamanho $2X2$ com apenas 7 operações, utilizando a estratégia de divisão e conquista aplicar a matrizes $N \times N$. O algoritmo surge da decomposição da forma:

$$XY = \sum_{k=1}^7 u_k(X)v_k(Y)W_k \quad (2)$$

Em que u_k e v_k são formas lineares escolhidas no espaço das matrizes $2X2$ e W_k são sete matrizes de tamanho $2X2$. Aplicando esse algoritmo é possível reduzir o custo computacional para $O(n^{\log_2 7})$ operações aritméticas [Ikenmeyer and Lysikov 2019].

2.7. Métricas de Desempenho

Entre as principais métricas para avaliação do desempenho de algoritmos destacam-se: Tempo de execução; *Overhead* paralelo total; *Speedup*; Eficiência;

[Grama et al. 2003] definem o tempo de execução de um algoritmo sequencial T_s como o tempo decorrido entre o início e o fim de sua execução em um computador sequencial. Já o tempo de execução de um algoritmo paralelo T_p é definido como o tempo decorrido a partir do momento em que a computação paralela se inicia até o momento da execução do último elemento de processamento.

Os custos computacionais adicionados devido à paralelização são encapsulados em uma expressão chamada *overhead*, que pode ser definida como o tempo total gasto por todos elementos coletivamente, além do tempo necessário pelo algoritmo sequencial [Grama et al. 2003]. Dessa forma, sendo p o número de *threads* *overhead* paralelo total pode ser calculado como:

$$T_o = pT_p - T_s \quad (3)$$

Speedup é a medida que mostra o benefício relativo de programar paralelamente em relação ao programa sequencial [Grama et al. 2003]. Assim, o *Speedup* do sistema pode ser calculado como a razão do tempo de execução sequencial pelo tempo em paralelo como na equação abaixo:

$$S_p = \frac{T_s}{T_p} \quad (4)$$

Já a eficiência é uma medida da fração de tempo na qual um elemento de processamento está utilmente empregado [Grama et al. 2003]. A eficiência pode ser calculada como:

$$E_p = \frac{T_s}{pT_p}. \quad (5)$$

3. Trabalhos Relacionados

3.1. Parallel matrix transpose algorithms on distributed memory concurrent computers

O artigo “Parallel matrix transpose algorithms on distributed memory concurrent computers” [Choi et al. 1995] trata da criação de um algoritmo referente à realização de operações de transposição de matrizes utilizando-se de computação paralela.

Para realizar o procedimento, o algoritmo separa os processadores disponíveis em uma matriz de duas dimensões e, utilizando distribuição de dados bloco-cíclica (block cyclic data distribution), divide sub-seções da matriz a serem analisadas por processadores específicos.

Dependendo da divisão dos processadores na representação 2D, sendo P o número de linhas e Q o número de colunas, duas situações distintas podem ocorrer: o máximo divisor comum (MDC) entre P e Q é calculado. Caso ele seja igual a 1, os números são matematicamente primos, e todos os processadores precisam realizar os procedimentos realizando ‘comunicação total’ entre si.

Caso o MDC seja maior que 1, o algoritmo divide os blocos de processadores em grupos delimitados pelo MDC, nos quais todas as operações de transposição entre blocos de mesmo grupo podem ocorrer simultaneamente, diminuindo a quantidade total de tempo de execução.

A análise final do algoritmo realizada no artigo demonstra que, em uma multiplicação de matrizes, transpor o resultado da operação utilizando o processo descrito aumenta em pouco o tempo total de execução. Ademais, o algoritmo se destaca, em principal, ao suportar versatilidade quanto a proporções em grades bidimensionais não quadradas (com P diferente de Q), não só mantendo alta eficiência, mas superando algoritmos restritos a arranjos quadrados.

Um ponto negativo observável no algoritmo, no entanto, possui uma descrição breve no artigo em si. Quando P e Q tem MDC igual a 1 e a comunicação total é realizada, a sua implementação é não só mais lenta do que o método com MDC maior, mas também não se utiliza da melhor implementação possível em teoria. Os autores optam por uma implementação não ótima que, dada a quantidade arbitrariamente grande de processadores, seria mais simples de ser utilizada.

3.2. Anatomy of High-Performance Matrix Multiplication

Outro artigo mais diretamente relacionado ao tópico é “Anatomy of High-Performance Matrix Multiplication” [Goto and Geijn 2008], que discorre diretamente sobre implementações otimizadas de operações de multiplicação de matriz, mais especificamente sobre gerenciar de maneira eficiente a hierarquia de memória disponível para obter alto desempenho de maneira consistente.

Os autores identificam que, nas abordagens mais tradicionais, os algoritmos criados não condizem às capacidades reais do hardware ao mensurar performance, como o número finito de entradas no Translation Look-aside Buffer (TLB) e as latências das memórias cache. O algoritmo proposto, portanto, busca segmentar mais o problema utilizando empacotamento a fim de aproveitar mais a memória no nível local.

Para evitar os gargalos de memória descritos, o sistema proposto reestrutura o algoritmo comum da GEMM (General Matrix Multiplication) em níveis distintos: no mais alto nível, a matriz de entrada é particionada de modo que o painel da matriz A é dimensionado para se ajustar ao cache L2 (ou L3) e ao limite de entradas do TLB, enquanto os blocos da matriz B são empacotados para fluir diretamente pelo cache L1.

Em nível de bloco, os dados lidos atribuídos passam pelo processo de empacotamento, a partir do qual os pacotes são inseridos em buffers temporários contíguos, para que o acesso a eles possa ser mais rápido e direto no processamento.

No micro-kernel, o processamento é realizado em pequenas submatrizes, maximizando a instrução vetorial e o paralelismo ao nível de instrução. O artigo apresenta variados experimentos que demonstram o aumento de desempenho dado por sua abordagem, e é uma base fundamental para análise referente às operações fundamentais de GEMM, e o algoritmo é utilizado em múltiplas bibliotecas atuais de computação de alta performance (HPC), como OpenBLAS e Intel MKL.

Apesar da alta eficiência do procedimento, um ponto negativo observável é que o processo é bastante elaborado e, portanto, contém overhead significativo. Dado que os procedimentos buscam utilizar o máximo possível de performance, o algoritmo é projetado para uma quantidade grande de processamento necessário e, caso a operação seja simples, as soluções implementadas seriam mais custosas do que ótimas em totalidade.

3.3. Relação com o Projeto Principal

Os artigos descritos, para o propósito deste projeto, apresentam não apenas algoritmos eficientes em suas respectivas áreas de ênfase (transposição e multiplicação de matrizes), mas também demonstram, ao longo de suas análises, diversas estratégias de gerenciamento de recursos do sistema para otimizar processos dependentes de matrizes. Dessa forma, mesmo que o presente trabalho não consista em uma reconstrução direta de nenhum dos métodos, ambos os artigos contêm esquemas de particionamento e análise bastante úteis para o desenvolvimento de nosso projeto.

4. Implementação

4.1. Implementação em blocos

A primeira abordagem implementada e avaliada foi o algoritmo de multiplicação de matrizes baseado na estratégia de divisão e conquista (*Divide and Conquer*), estruturado de forma recursiva para o particionamento de submatrizes. Essa técnica foi selecionada por permitir uma divisão estruturada do problema, cujos casos bases são resolvidos por meio de uma rotina paralela customizada que integra diretivas OpenMP e otimização vetorial SIMD.

O código foi desenvolvido na linguagem C++ para poder utilizar templates para os tipos de dados e emprega a biblioteca OpenMP para a paralelização do processamento. Como o particionamento recursivo pode introduzir um custo elevado de gerência de tarefas, implementou-se um parâmetro limiar de corte (k_c), garantindo que o paralelismo e a vetorização sejam aplicados de forma equilibrada somente quando as submatrizes atingem dimensões adequadas. Adicionalmente, o framework de testes realiza comparações diretas de exatidão e desempenho com as rotinas altamente otimizadas da biblioteca OpenBLAS (*cblas_sgemv* para precisão simples e *cblas_dgemv* para precisão dupla).

Os testes foram conduzidos considerando diferentes dimensões de matrizes quadradas (1000x1000, 2000x2000, 3000x3000 e 4000x4000) e uma variação ampla de valores para o limiar k_c . As matrizes de teste são populadas com valores em ponto flutuante gerados aleatoriamente (nos tipos *float* e *double*), utilizando alinhamento de memória específico para favorecer o desempenho das instruções de hardware.

As métricas de desempenho coletadas e analisadas compreendem o tempo de execução sequencial (T_1), o tempo de execução paralelo (T_P), o *speedup*, a eficiência paralela, o *overhead* paralelo total e o rendimento computacional medido em GFLOPS, permitindo confrontar a solução desenvolvida com os padrões industriais do OpenBLAS.

4.2. Método de Strassen

A segunda implementação escolhida para os testes foi o algoritmo de Strassen para multiplicação de matrizes. Esse algoritmo foi escolhido pois possui algumas propriedades que tornam sua paralelização um caso interessante. Como discutido anteriormente na fundamentação, esse algoritmo efetua multiplicações em matrizes 2x2 utilizando somente 7 operações, isso é interessante devido a essas operações serem independentes entre si, possibilitando a paralelização de sua execução para verificação da eficiência.

As métricas de desempenho utilizadas nessa implementação são as mesmas vistas anteriormente, tempo de execução tanto sequencial quanto paralelo, *overhead* paralelo total, *speedup* e eficiência.

O código foi implementado na linguagem C e utiliza o OpenMP para efetuar sua paralelização. *Multithreading* é utilizado para realizar as operações de soma e subtração de matrizes existentes no método de Strassen. Entretanto, algumas restrições foram implementadas para impedir que o custo de gerenciar todas as tarefas existentes não supere o ganho do paralelismo. Como o método de Strassen utiliza recursão, a criação desregulada de tarefas pode rapidamente diminuir a eficiência do programa, por conta disso, uma restrição foi colocada para que novas tarefas sejam criadas somente quando a matriz sendo computada seja de um certo tamanho.

As matrizes utilizadas são geradas de maneira aleatória com dimensão $N \times N$ determinada em tempo de execução e cada elemento é povoado com um valor aleatório do tipo *int* com valor variando entre -1000 e 1000.

5. Resultados

5.1. Implementação em blocos

Os testes relacionados à multiplicação de matrizes por blocos foram realizados com 4 tamanhos de matrizes diferentes: 1000x1000, 2000x2000, 3000x3000 e 4000x4000, utilizando 16 threads ativas e variando o tamanho do bloco (k_c) para os tipos de dados *float* e *double*. Aqui será apresentado somente as matrizes 1000x1000 e 4000x4000 devido à limitação de espaço. O campo *Ref: cblas_sgemv* contém os parâmetros de referência para a execução utilizando a biblioteca OpenBLAS.

A primeira parte da Tabela 1 apresenta os resultados obtidos para a matriz 1000x1000. Nota-se que tamanhos de blocos muito pequenos ($k_c = 64$ e 96) resultam em um desempenho reduzido e em um elevado *overhead* devido à granulação inadequada. Conforme o tamanho do bloco aumenta, o desempenho melhora expressivamente, alcançando um *speedup* de até 6.81x para *float* ($k_c = 1000$) e 7.41x para *double* ($k_c = 1000$).

A segunda parte da Tabela 1 apresenta os testes realizados com matrizes 4000x4000. Neste cenário de maior carga de trabalho, a eficiência máxima ultrapassa 54% no formato *double* ($k_c = 2000$), com um *speedup* superior a 8.6x. Os dados comprovam que a escolha adequada do parâmetro de bloco k_c é determinante para mitigar o *overhead* de sincronização e maximizar a eficiência do paralelismo em larga escala.

Tabela 1. Resultados da implementação em blocos para matrizes de 1000x1000 e 2000x2000

Matriz	Tipo	k_c	Tempo (s)	GFLOPS	T_1 (1 thd)	Overhead (s)	Speedup	Eficiência
Matriz 1000x1000 Ref: cblas_sgemv (0.0060s, 333.33 GFLOPS)								
1000x1000	float	64	2.3710	0.84	2.2750	35.6610	0.96 x	6.00 %
1000x1000	float	96	2.4080	0.83	2.4100	36.1180	1.00 x	6.26 %
1000x1000	float	128	0.7740	2.58	2.4180	9.9660	3.12 x	19.53 %
1000x1000	float	192	0.5230	3.82	2.5550	5.8130	4.89 x	30.53 %
1000x1000	float	256	0.4040	4.95	2.3990	4.0650	5.94 x	37.11 %
1000x1000	float	384	0.4140	4.83	2.3220	4.3020	5.61 x	35.05 %
1000x1000	float	512	0.3620	5.52	2.4250	3.3670	6.70 x	41.87 %
1000x1000	float	1000	0.3270	6.12	2.2260	3.0060	6.81 x	42.55 %
Matriz 1000x1000 Ref: cblas_dgemv (0.0250s, 80.00 GFLOPS)								
1000x1000	double	64	4.2530	0.47	2.5380	65.5100	0.60 x	3.73 %
1000x1000	double	96	4.7670	0.42	2.5790	73.6930	0.54 x	3.38 %
1000x1000	double	128	0.8150	2.45	2.4620	10.5780	3.02 x	18.88 %
1000x1000	double	192	0.7640	2.62	2.7290	9.4950	3.57 x	22.32 %
1000x1000	double	256	0.4180	4.78	2.5620	4.1260	6.13 x	38.31 %
1000x1000	double	384	0.4340	4.61	2.4850	4.4590	5.73 x	35.79 %
1000x1000	double	512	0.3640	5.49	2.5030	3.3210	6.88 x	42.98 %
1000x1000	double	1000	0.3330	6.01	2.4660	2.8620	7.41 x	46.28 %
Matriz 4000x4000 Ref: cblas_sgemv (0.2150s, 595.35 GFLOPS)								
4000x4000	float	64	34.4670	3.71	144.4000	407.0720	4.19 x	26.18 %
4000x4000	float	96	35.2220	3.63	144.2500	419.3020	4.10 x	25.60 %
4000x4000	float	128	20.8460	6.14	140.4520	193.0840	6.74 x	42.11 %
4000x4000	float	192	21.1480	6.05	139.5450	198.8230	6.60 x	41.24 %
4000x4000	float	256	20.3230	6.30	140.2450	184.9230	6.90 x	43.13 %
4000x4000	float	384	19.3510	6.61	140.9610	168.6550	7.28 x	45.53 %
4000x4000	float	512	19.1570	6.68	140.2440	166.2680	7.32 x	45.75 %
4000x4000	float	1000	18.4750	6.93	138.5720	157.0280	7.50 x	46.88 %
4000x4000	float	2000	18.5030	6.92	139.0880	156.9600	7.52 x	46.98 %
4000x4000	float	3000	18.1020	7.07	140.2420	149.3900	7.75 x	48.42 %
4000x4000	float	4000	18.3910	6.96	138.2450	156.0110	7.52 x	46.98 %
Matriz 4000x4000 Ref: cblas_dgemv (0.4660s, 274.68 GFLOPS)								
4000x4000	double	64	35.4760	3.61	162.5030	405.1130	4.58 x	28.63 %
4000x4000	double	96	35.5220	3.60	162.8520	405.5000	4.58 x	28.65 %
4000x4000	double	128	21.3970	5.98	160.9620	181.3900	7.52 x	47.02 %
4000x4000	double	192	21.9600	5.83	160.9740	190.3860	7.33 x	45.81 %
4000x4000	double	256	19.6170	6.52	161.8420	152.0300	8.25 x	51.56 %
4000x4000	double	384	19.5050	6.56	162.1140	149.9660	8.31 x	51.95 %
4000x4000	double	512	18.8900	6.78	161.1970	141.0430	8.53 x	53.33 %
4000x4000	double	1000	18.7940	6.81	161.0110	139.6930	8.57 x	53.54 %
4000x4000	double	2000	18.7800	6.82	162.5620	137.9180	8.66 x	54.10 %
4000x4000	double	3000	18.7810	6.82	162.5310	137.9650	8.65 x	54.09 %
4000x4000	double	4000	18.9490	6.75	161.1390	142.0450	8.50 x	53.15 %

5.2. Método de Strassen

Os testes relacionados ao método de Strassen foram realizados com 3 tamanhos de matrizes diferentes: 100x100, 200x200 e 300x300. Além de 4 quantidades diferentes de threads: 2, 4, 8 e 16.

A figura 1 abaixo mostra os resultados obtidos na multiplicação de matrizes 100x100. É possível perceber que houve um ganho significativo de desempenho ao aumentar o número de threads até 8. Esse é o resultado esperado, visto que a implementação do algoritmo gera 7 tarefas. Como existe uma restrição sobre a criação de novas tarefas relacionada ao tamanho da matriz, o caso com 16 threads não encontrou tarefas suficientes, além de sofrer com o custo do gerenciamento de 16 threads simultâneas, tornando o resultado menos eficiente que o com 8 threads.

	Matriz 100 x 100				
Threads	T Sequencial	T paralelo	Overhead Paralelo	Speedup	Eficiência
2	0.161s	0.093s	-0.068s	1.731x	86.56%
4	0.165s	0.048s	-0.117s	3.437x	85.94%
8	0.161s	0.028s	-0.133s	5.75x	71.87%
16	0.166s	0.036s	-0.13s	4.611x	28.82%

Figura 1. Resultados do paralelismo em matrizes 100x100

A figura 2 abaixo mostra os resultados da multiplicação para uma matriz 200x200. Com uma matriz maior, a perda de eficiência do caso com 16 threads foi inferior ao caso anterior, havendo um pequeno ganho de eficiência em comparação com o caso de 8 threads. Entretanto, apesar do *speedup* superior, a eficiência ainda é baixa, devido ao custo de manutenção de todas as threads.

	Matriz 200 x 200				
Threads	T Sequencial	T paralelo	Overhead Paralelo	Speedup	Eficiência
2	1.12s	0.564s	-0.556s	1.986x	99.29%
4	1.185s	0.311s	-0.874s	3.810x	95.26%
8	1.1s	0.197s	-0.911s	5.624x	70.3%
16	1.147s	0.171s	-0.976s	6.708x	41.92%

Figura 2. Resultados do paralelismo em matrizes 200x200

Já na figura 3, com uma matriz ainda maior de 300x300, houve novamente um ganho de eficiência para todos os casos, especialmente os casos com 8 e 16 threads. Esse é o resultado esperado e mostra o potencial do paralelismo em casos de multiplicação de matrizes grandes. Apesar da eficiência do caso de 16 threads ainda ser baixa, ele mostrou um *speedup* de mais de 7x, tornando uma execução sequencial de quase 8 segundos em uma execução com paralelismo em pouco mais de 1 segundo.

	Matriz 300 x 300				
Threads	T Sequencial	T paralelo	Overhead Paralelo	Speedup	Eficiência
2	7.86s	3.943s	-3.925s	1.995x	99.77%
4	8.487s	2.281s	-6.2s	3.721x	93.02%
8	7.753s	1.27s	-6.483s	6.105x	76.31%
16	7.942s	1.079s	-6.863s	7.361x	46%

Figura 3. Resultados do paralelismo em matrizes 300x300

Os resultados apresentados mostram, como um todo, que o paralelismo é muito bem aplicado no algoritmo de Strassen, em todos os casos houve um overhead paralelo negativo, mostrando um tempo de execução mais curto para a implementação paralela. Os testes efetuados mostram que o uso de 8 threads gera os resultados mais balanceados entre performance e eficiência, um resultado esperado, visto que o método de Strassen foi paralelizado em suas 7 operações em matrizes independentes.

6. Conclusão

Após a implementação e teste de dois diferentes algoritmos de multiplicação de matrizes, é possível concluir que existe ganho significativo de desempenho para esses algoritmos por meio da execução paralela. Tanto o algoritmo de blocos quanto o algoritmo de Strassen apresentaram um ganho de desempenho significativo, sempre intensificado pelo tamanho das matrizes, com os maiores ganhos ocorrendo em matrizes maiores.

Os testes com diversas quantidades de threads do algoritmo de Strassen mostram que, para esse algoritmo, o uso de 8 threads atinge um equilíbrio entre speedup e eficiência, obtendo melhorias significativas no tempo de execução, além de seguirem o padrão visto anteriormente de aumento de eficiência baseado no tamanho da matriz. A implementação do paralelismo para as 7 operações de matrizes existentes e a grande eficiência encontrada no uso de 8 threads são resultados encorajadores que mostram o potencial dessas implementações.

Os resultados na implementação por blocos demonstram também um forte ganho de eficiência com o uso de paralelismo, com valores de speedup próximos a 8x em algumas instâncias de float e próximo de 9x em algumas instâncias de double. Apesar de alguns problemas de eficiência em casos com blocos pequenos, a estabilização e o ganho de desempenho em blocos maiores mostra o potencial do paralelismo nesse caso também.

Referências

- [Allen and Kennedy 2002] Allen, R. and Kennedy, K. (2002). *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. Morgan Kaufmann, San Francisco.
- [Choi et al. 1995] Choi, J., Dongarra, J. J., and Walker, D. W. (1995). Parallel matrix transpose algorithms on distributed memory concurrent computers. *Parallel Computing*, 21(9):1387–1405.
- [Cormen et al. 2012] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2012). *Algoritmos: teoria e prática*. Elsevier, Rio de Janeiro, 3a edition.

- [Cppreference.com 2026] Cppreference.com (2026). Array declaration. Acesso em: 26 set. 2026.
- [Goto and Geijn 2008] Goto, K. and Geijn, R. A. v. d. (2008). Anatomy of high-performance matrix multiplication. 34(3).
- [Grama et al. 2003] Grama, A., Gupta, A., Karypis, G., and Kumar, V. (2003). *Introduction to Parallel Computing*. Addison-Wesley/Pearson Education, Harlow, 2nd edition.
- [Hennessy and Patterson 2017] Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition.
- [Herlihy and Shavit 2008] Herlihy, M. and Shavit, N. (2008). *The Art of Multiprocessor Programming*. Morgan Kaufmann, Boston.
- [Ikenmeyer and Lysikov 2019] Ikenmeyer, C. and Lysikov, V. (2019). Strassen's 2×2 matrix multiplication algorithm: A conceptual perspective. *Annali dell'Universita di Ferrara*, 65(2):241–248.
- [Intel Corporation 2023] Intel Corporation (2023). Vectorization.
- [Pacheco 2011] Pacheco, P. S. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.